

CERYNTHOS

Linux-First Alpha Execution Plan

A focused 12-week roadmap for a four-person team



LINUX-FIRST DELIVERY STRATEGY

CerynthOS Alpha Execution Plan

A focused, feasible roadmap for a four-person team

DELIVERY WINDOW	TEAM SIZE	PRIMARY STACK
12 weeks	4 people	Linux + Rust

1. Executive direction

The decision

Build a real Linux-based CerynthOS first, prove the adaptive ideas on real workloads, and keep the from-scratch kernel as a later parallel research track.

This scope change preserves the ambitious long-term vision while giving the team a credible near-term product. The first release is not merely a themed Linux distribution: it is a bootable, installable operating system with a Rust-based adaptive scheduling layer, local telemetry, safe profile selection, and an AI-first control experience.

The three-step strategy

Stage	Focus	Outcome
1	Build a reliable Linux OS	Own the system image, desktop identity, installer, release process, and recovery path.
2	Make it adaptive	Use sched_ext, Rust scheduler policies, workload classification, shadow mode, and rollback.
3	Make it intelligent	Expose safe OS actions through an AI command surface and explain system behaviour to the user.

Alpha delivery target

- A bootable and installable CerynthOS ISO.
- A distinct desktop, onboarding flow, Control Center, and release identity.
- A recent Linux kernel configured for sched_ext.
- A Rust-based scheduler with at least four safe profiles.
- Automatic workload classification and profile selection.
- Shadow mode, regression detection, health monitoring, and immediate fallback.
- A limited AI command interface for real, permission-checked system actions.
- Published benchmarks, documentation, and reproducible builds.

SYSTEM SHAPE

2. Product and technical architecture

CerynthOS Alpha - Linux-first architecture

Cerynth Experience Layer

- KDE Plasma desktop and Cerynth visual identity
- Control Center, onboarding, performance dashboard
- AI command palette with typed, permission-checked actions

Cerynth System Layer - primarily Rust

- cerynthd daemon and cerynthctl command-line utility
- Telemetry, workload classification, privacy controls
- Adaptive profile selection, rollback and local system API

Linux Foundation Layer

- Recent Linux kernel with sched_ext and eBPF integration
- Rust scheduler policy through scx_rustland_core
- Automatic fallback to the normal Linux scheduler

Recommended implementation choices

Area	Choice	Reason
Base distribution	Fedora- or Arch-family base	Recent kernels, strong developer tooling, and practical sched_ext access.
Desktop	KDE Plasma	Highly customisable without building a compositor or shell from scratch.
Core services	Rust	Memory safety, good systems tooling, and alignment with the scheduler work.
Kernel integration	sched_ext + eBPF	Runtime-loadable scheduling policy with kernel-enforced safety and fallback.
Scheduler bridge	scx_rustland_core	Keeps most policy logic in Rust userspace while using the real Linux scheduler interface.
Training and analysis	Python	Fast experimentation for trace analysis and small classifiers.
UI	QML / KDE integration	Fits Plasma and allows a polished native Control Center experience.

Language boundary

Rust is the primary project language. Small amounts of BPF C are acceptable for kernel-side hooks, and Python is used only for offline experiments and data analysis. Do not force every component into Rust when doing so adds risk without product value.

3. Four-person team structure

Member 1	Platform and Release
Responsibilities Base distribution; kernel configuration; ISO; installer; package repository; build automation; updates; rollback; release images.	Primary outcome CerynthOS installs, boots, updates, and recovers reliably.
Member 2	Scheduler and Kernel Integration
Responsibilities sched_ext; scx_rustland_core; eBPF bridge; Rust scheduler; profiles; watchdogs; fallback; scheduler benchmarks.	Primary outcome CerynthOS safely changes CPU scheduling behaviour on real workloads.
Member 3	Adaptive Intelligence
Responsibilities Telemetry schema; workload classification; behaviour history; adaptive profile selection; evaluation; privacy controls.	Primary outcome The OS learns which vetted policy works best for the current workload.
Member 4	Desktop and AI Experience
Responsibilities Theme; Control Center; onboarding; scheduler controls; dashboard; AI command palette; user testing.	Primary outcome The system feels like a coherent operating system, not a research bundle.

Team rule

Every subsystem has one clear owner. Everyone contributes to testing and documentation, but architectural ownership must not be shared ambiguously.

4. Twelve-week delivery roadmap

Pace

Twelve weeks is intentionally aggressive. The team should freeze the alpha scope at the end of Week 1 and reject additions unless they replace an existing item.

Week	Phase	Required result
1	Foundation	Scope, common environment, CI, base image
2	First OS image	Bootable branded desktop and installer path
3-4	Scheduler foundation	Rust scheduler, profiles, fallback, baselines
5-6	System services	cerynthd, cerynthctl, API, controls, telemetry
7-8	Adaptive MVP	Classifier, automatic switching, shadow mode, rollback
9-10	AI-first experience	Command palette, explanations, safe actions
11	Stability	Real hardware, stress, suspend, drivers, recovery
12	Alpha release	ISO, demo, benchmark report, docs, v0.1-alpha

Week 1 - Scope and development environment

- Freeze the alpha feature list and select one base distribution.
- Create repositories, branching rules, issue labels, and ownership boundaries.
- Set up CI, reproducible builds, VM testing, and the sched_ext development environment.
- Define Rust style, telemetry schema versioning, local API conventions, and release naming.

Exit gate

All four developers can build and boot the same development image.

Week 2 - First CerynthOS image

- Generate a bootable ISO and a reproducible image build.
- Add CerynthOS naming, boot splash, desktop theme, and default applications.
- Configure the installer path and automated nightly images.
- Include baseline debugging and performance tools.

Exit gate

CerynthOS boots into its own branded desktop in a VM.

Weeks 3-4 - Rust scheduler foundation

- Run existing sched_ext schedulers and record EEVDF and SCX baselines.
- Build the first Rust scheduler using scx_rustland_core.
- Implement Balanced, Interactive, Performance, and Background profiles.
- Add live profile switching, scheduler health reporting, logs, and automatic fallback.

Exit gate

Profiles switch live, and a policy failure safely returns the system to Linux scheduling.

Weeks 5-6 - Cerynth system services

- Build cerynthd to manage scheduler state, telemetry, health, privacy, and local API access.
- Build cerynthctl for diagnostics, profile control, reset, and safe recovery.
- Create the first Control Center pages and a system tray status component.
- Add private mode, learning pause, and personalisation-data reset.

Exit gate

A user can control the adaptive system without opening a terminal.

Weeks 7-8 - Adaptive scheduling MVP

- Classify interactive, gaming, compilation, batch, idle, and mixed workloads.
- Start with rules and statistics; introduce a small decision tree only when traces justify it.
- Add confidence thresholds, switching cooldowns, shadow mode, regression detection, and rollback.
- Explain why a profile was selected and allow immediate manual override.

Exit gate

CerynthOS automatically switches among verified profiles without sacrificing safety.

Weeks 9-10 - AI-first experience

- Build a focused command palette rather than a general unrestricted agent.
- Translate natural language into typed, permission-checked system intents.
- Support performance questions, profile changes, learning controls, and system summaries.
- Add confirmation for sensitive actions and never grant unrestricted root or shell access.

Exit gate

The AI interface reliably performs a small set of real operating-system actions.

Week 11 - Stability and hardware testing

- Test multiple laptops or desktops, suspend and resume, audio, networking, Bluetooth, and graphics.
- Run long scheduler stress tests and repeated failure-recovery scenarios.
- Measure idle CPU, memory overhead, scheduler latency, and log volume.
- Fix installation, update, recovery, and permission defects.

Exit gate

The OS survives normal daily use and scheduler failures without reinstallation.

Week 12 - Alpha release

- Produce and sign the release ISO and publish installation instructions.
- Record a demo and publish benchmark results with honest limitations.
- Publish architecture, contribution, recovery, privacy, and known-issues documentation.
- Tag v0.1-alpha and freeze the next milestone backlog.

Exit gate

A public, installable CerynthOS alpha is reproducible from the repository.

RESEARCH FOCUS

5. Adaptive scheduler MVP

The first adaptive release should not begin with reinforcement learning. It should select among reliable, already-tested scheduling profiles using lightweight evidence from the current workload. This gives the team a demonstrable learning loop without allowing an unverified model to control the machine.

Initial workload classes

Workload	Observable pattern	Suggested policy
Interactive desktop	Frequent wakeups, short CPU bursts, foreground input activity	Interactive profile
Gaming / latency sensitive	Stable foreground process, frame-sensitive activity, background contention	Performance or latency profile
Compilation	Many CPU-bound worker processes and sustained run queue depth	Throughput profile
Background processing	CPU-heavy tasks without foreground interaction	Background / fair-share profile
Idle / power saving	Low activity and long idle windows	Balanced or energy-saving profile
Mixed workload	Foreground latency-sensitive work plus batch contention	Interactive profile with bounded batch allocation

Safety and adaptation loop

1. Observe compact workload features locally.
2. Classify the current workload and calculate confidence.
3. Choose only from vetted scheduling profiles.
4. Run uncertain changes in shadow mode before promotion.
5. Apply cooldowns to prevent rapid profile oscillation.
6. Monitor latency, throughput, fairness, and scheduler health.
7. Rollback immediately when a regression or watchdog condition is detected.
8. Explain the decision and preserve manual user control.

Scope inherited from MIMIR

Keep telemetry, safe policy selection, behavioural-cloning foundations, fallback, and future model-zoo compatibility. Postpone offline RL, DAgger, model-to-BPF compilation, and continuous retraining until the alpha is stable.

USER LAYER

6. AI-first experience

The first AI interface should be a system command and explanation layer. It does not need to be a general assistant. Its job is to make operating-system behaviour understandable and controllable.

Example request	Typed system action
Switch to performance mode.	Set a known profile through cerynthd.
Why is my computer slow?	Summarise CPU, memory, I/O, thermal, and scheduler observations.
Which applications are using the CPU?	Display a ranked process summary with context.
Optimise the system for this game.	Apply a verified latency-sensitive profile and monitor it.
Pause learning for one hour.	Disable collection and automatic changes for a bounded period.
What has CerynthOS learned?	Show inspectable local preferences and workload classifications.
Reset my personalisation data.	Delete stored adaptation state after explicit confirmation.

Non-negotiable safety rules

- No unrestricted root access for the AI interface.
- No raw arbitrary shell execution from natural language.
- Every operation maps to a typed intent with explicit permissions.
- Sensitive changes require confirmation and produce an activity record.
- The user can inspect, pause, override, and reset all personalisation.
- Local-first data handling is the default.

WHAT MUST BE DONE

7. Master implementation checklist

No.	Task	Owner
1	Select and configure the Linux distribution base.	Platform
2	Build an automated ISO and release pipeline.	Platform
3	Create the CerynthOS visual identity and desktop defaults.	Experience
4	Enable and validate sched_ext on the chosen kernel.	Scheduler

No.	Task	Owner
5	Build the Rust scheduler and eBPF integration seam.	Scheduler
6	Implement four safe scheduling profiles.	Scheduler
7	Build cerynthd and cerynthctl.	Adaptive
8	Collect lightweight local workload telemetry.	Adaptive
9	Build the workload classifier and adaptive profile selector.	Adaptive
10	Add shadow mode, confidence thresholds, cooldowns, and rollback.	Adaptive
11	Build the Cerynth Control Center and performance dashboard.	Experience
12	Add the AI command and explanation interface.	Experience
13	Add installer, update, rollback, and recovery paths.	Platform
14	Benchmark against the normal Linux scheduler and strong SCX baselines.	Scheduler + Adaptive
15	Test on real hardware and publish known limitations.	All
16	Release the alpha with reproducible documentation.	All

PROTECT THE DEADLINE

8. Explicitly postponed scope

These ideas remain part of the longer CerynthOS vision, but they should not enter the 12-week alpha unless an existing commitment is removed.

NOT IN ALPHA From-scratch kernel and boot chain	NOT IN ALPHA Custom package manager
NOT IN ALPHA Custom desktop environment or compositor	NOT IN ALPHA An original foundational AI model
NOT IN ALPHA Live reinforcement learning in the scheduler	NOT IN ALPHA Neural-network inference on every scheduling event
NOT IN ALPHA Model-to-BPF compiler	NOT IN ALPHA Automatic kernel modification
NOT IN ALPHA Learned memory, I/O, and network policies	NOT IN ALPHA Universal hardware compatibility
NOT IN ALPHA ARM, mobile, and embedded targets	NOT IN ALPHA Enterprise fleet learning
NOT IN ALPHA Fully autonomous system agents	

RELEASE GATE

9. Alpha success criteria

Gate	Definition	Pass condition
Installable	Boots and installs from a CerynthOS ISO.	Required
Distinct	Has a coherent desktop identity, onboarding, and system controls.	Required
Real scheduling	Runs a Rust scheduling policy on real Linux workloads.	Required
Multiple policies	Switches among at least four verified profiles.	Required
Adaptive	Automatically selects profiles using local workload evidence.	Required
Safe	Falls back immediately after policy failure or measurable regression.	Required
Transparent	Explains selections and lets the user pause, override, and reset learning.	Required
AI-first	Executes a small, reliable set of typed operating-system actions.	Required
Measured	Shows at least one meaningful workload win without unacceptable regressions.	Required
Reproducible	Builds from a clean repository using documented steps.	Required
Operating principle First build a reliable Linux OS. Then make it adaptive. Then make it intelligent. Later, replace the kernel.		

FIRST FIVE WORKING DAYS

10. Immediate kickoff checklist

Day	Required output
Day 1	Freeze alpha scope; choose base distribution; assign subsystem owners; create repositories.
Day 2	Prove reproducible VM image build; enable sched_ext; capture baseline kernel and hardware configuration.
Day 3	Boot an existing SCX scheduler; begin ISO branding; define cerynthd API and telemetry event schema.
Day 4	Create the Rust scheduler skeleton; create Control Center wireframes; set up benchmark automation.
Day 5	Demo the first integrated vertical slice: Cerynth-branded VM, SCX policy control, basic status UI, CI artifact.

Decision log for Week 1

- Base distribution and kernel version.
- Desktop environment and installer approach.
- Exact scheduler integration repository or fork.
- Telemetry fields permitted in the alpha.
- Benchmark machines and baseline workloads.
- Release naming, supported hardware statement, and privacy defaults.

SOURCES

References and technical starting points

- Linux kernel documentation: Extensible Scheduler Class (sched_ext).
- sched-ext/scx repository, including scx_rustland and scx_rustland_core.
- MIMIR - A Learned, Self-Improving CPU Scheduler for Linux, design and research proposal v0.1.
- Linux eBPF and BPF CO-RE documentation.
- KDE Plasma and QML development documentation.

CERYNTHOS

Build the system. Prove the adaptation. Earn the custom kernel.